

KSU CONNECT

Project Documentation

A Scalable Social Media Platform for Connection, Expression & Community

Document type:	Project Architecture & Technical Specification
Application:	KSU CONNECT – Social Media Web Application
Platform:	Web (React / Next.js) + Mobile (React Native)
Architecture:	Microservices · REST API · Event-driven
Version:	v1.0
Status:	Technical Draft

Table of Contents

1.	Executive Summary	3
2.	Project Overview	3
3.	Core Features & Requirements	4
4.	System Architecture	6
5.	Technology Stack	7
6.	Database Design	8
7.	API Design	10
8.	Security Considerations	12
9.	Development Roadmap	13
10.	Suggested Team Structure	13
11.	Conclusion	14

1. Executive Summary

KSU CONNECT is a full-featured social media web application designed to foster connection, expression, and community. Inspired by platforms like Facebook and Instagram, it enables users to create personal profiles, share images and videos, engage through likes and comments, and build social networks through a follow system. The platform is built with scalability, security, and a seamless experience at its core.

This document provides a comprehensive technical specification covering system architecture, technology stack, database schema, API endpoints, security considerations, and a phased development roadmap for teams planning and building KSU CONNECT.

2. Project Overview

2.1 Purpose

KSU CONNECT provides a dedicated digital space where users can connect, share experiences, and engage with content in a safe, scalable, and performant environment. The platform prioritises authentic social interaction backed by robust privacy controls and moderation tools.

2.2 Goals

- Build a highly scalable social platform capable of supporting thousands of concurrent users.
- Deliver a seamless, responsive experience across web and mobile.
- Enable rich content sharing — photos, videos, and text posts.
- Implement robust social interactions: likes, comments, shares, and follows.
- Ensure data privacy, security, and best-practice compliance (GDPR).
- Deliver real-time notifications and live feed updates via WebSockets.
- Provide a content discovery engine powered by search and trending algorithms.

2.3 Target Users

KSU CONNECT targets individuals aged 16–45 who are social-media savvy and seeking a community-driven platform. Primary use cases include personal expression, networking, following interests, and community engagement. Secondary use cases include event promotion and group-based content sharing.

3. Core Features & Requirements

User Registration & Authentication

- Email / password registration and secure login
- OAuth2 social login (Google, Facebook)
- JWT-based stateless session management (access + refresh tokens)
- Email verification on registration and password-reset flow
- Optional two-factor authentication (TOTP / SMS)

User Profiles

- Customisable profile photo and cover banner image
- Bio, location, website, and pronouns fields
- Live post count, followers count, and following count
- Privacy toggle: public profile or private (follow-request required)
- Full account-settings panel (edit, deactivate, delete)

Post Creation & Management

- Text posts with up to 2 000 characters and rich emoji support
- Single or multi-photo uploads (JPEG, PNG, WebP, GIF)
- Video uploads up to 500 MB (MP4, MOV, AVI)
- Automatic server-side video transcoding via FFmpeg to HLS adaptive streaming
- Post caption, hashtags, and optional location tag
- Edit caption and visibility; permanently delete own posts
- Visibility options: Public, Followers only, or Private

Social Interactions

- Like / unlike posts with instant count update
- Threaded comment sections with nested replies
- Like individual comments
- Share / repost content to own feed with optional quote caption
- Emoji reactions (Like, Love, Haha, Wow, Sad, Angry)
- Mention users with @username inside captions and comments

Follow System

- Follow and unfollow any public user with a single tap
- Follow-request queue for private accounts; accept or decline
- View full followers and following lists for any profile
- Suggested users surfaced by mutual connections and shared interests
- Remove a follower from own account without blocking

News Feed

- Personalised chronological and ranked feed from followed accounts
- Explore / Trending tab with content beyond the follow graph
- Algorithmic ranking based on recency, engagement, and affinity
- Infinite scroll with cursor-based pagination and lazy-loaded media
- Stories — ephemeral full-screen content that disappears after 24 hours

Notifications

- Real-time in-app notification bell via WebSocket push
- Browser push notifications and mobile push (FCM / APNs)
- Alerts for: likes, comments, mentions, new followers, follow requests
- Granular notification-preference settings per event type
- Mark all as read and clear notification history

Search & Discovery

- Full-text search for users (username, full name) and post content
- Hashtag search and clickable hashtag links inside posts
- Trending hashtags and trending posts updated every 15 minutes
- Live search autocomplete with debounce
- Explore page with curated topic feeds and recommended creators

Privacy & Safety

- Block and report users or individual posts
- Automated content moderation via AWS Rekognition (explicit material)
- Human-review queue for flagged content; escalation workflow
- GDPR data-export (JSON) and permanent account-deletion within 30 days
- Cookie consent banner and Privacy Policy on first visit

4. System Architecture

KSU CONNECT adopts a microservices-based architecture organised into six logical layers. Each layer has a distinct concern, enabling independent scaling, deployment, and maintenance. Services communicate via REST (synchronous) and a message queue (asynchronous).

4.1 Architecture Layers

Layer	Components	Responsibility
Client layer	React / Next.js · React Native	UI rendering, client-side state, user interaction
CDN + Gateway	CloudFront CDN · API Gateway · Nginx	Static delivery, rate limiting, auth middleware, request routing
Backend services	Auth · User · Post · Media · Feed · Notification · Business Logic	Business logic split across independently deployable microservices
Real-time layer	Socket.io WebSocket · RabbitMQ / Kafka	Event-driven messaging, live notifications, feed fan-out
Data layer	PostgreSQL · Redis · Elasticsearch · AWS S3 · MongoDB	Persistent storage, caching, search indexing, media objects
Infrastructure	Docker · Kubernetes (EKS) · GitHub Actions	Containerisation, orchestration, automated testing and deployment

4.2 Microservice Descriptions

Auth Service

Manages user authentication via JWT (access 15 min / refresh 30 days) and OAuth2 (Google, Facebook). Session state is cached in Redis. Every request to the API Gateway is validated against the Auth Service before reaching downstream services.

User Service

Owns user profiles, account settings, privacy flags, and the social graph (followers / following). Stores relational data in PostgreSQL and exposes REST endpoints for profile retrieval, editing, and follow/unfollow operations.

Post Service

Handles full CRUD for posts, comments, likes, and reactions. On post creation it publishes a PostCreated event to the message queue, triggering Feed fan-out and Notification dispatch asynchronously, keeping the response latency low for the client.

Media Service

Receives multipart file uploads, validates MIME type and file size, stores original files in AWS S3, and triggers an FFmpeg transcoding pipeline for videos (HLS multi-bitrate). All media is served globally via CloudFront CDN with signed URLs.

Feed Service

Generates personalised newsfeeds using a hybrid push-pull strategy. Hot feeds for active users are pre-computed and stored in Redis. Cold/stale feeds are assembled on demand from PostgreSQL. Ranking signals include recency, engagement rate, and user affinity.

Notification Service

Consumes notification events from the message queue and delivers them in real-time via Socket.io (in-app) and via FCM / APNs (push). Notification history is persisted in MongoDB for fast document-based

retrieval.

Search Service

Maintains an Elasticsearch index synchronised with user and post data via CDC (Change Data Capture).

Powers full-text search, hashtag queries, and the explore discovery feed with sub-100 ms response times.

5. Technology Stack

Category	Technology	Purpose
Frontend	React.js / Next.js	Web UI with SSR/SSG for SEO and performance
Frontend	React Native	Cross-platform mobile app (iOS & Android)
Frontend	TypeScript	Static typing across the full frontend codebase
Frontend	Tailwind CSS	Utility-first responsive styling and design system
Frontend	Redux Toolkit	Global state — auth tokens, feed cache, notifications
Backend	Node.js + Express	REST API runtime for all microservices
Backend	GraphQL (Apollo Server)	Flexible query interface for complex data relations
Backend	JWT + OAuth2	Stateless auth, social login (Google, Facebook)
Backend	Socket.io	Real-time bidirectional WebSocket communication
Backend	FFmpeg	Server-side video transcoding to HLS / DASH
Backend	RabbitMQ / Apache Kafka	Async event bus for inter-service messaging
Database	PostgreSQL 16	Primary relational DB — users, posts, comments
Database	Redis 7	Session cache, feed cache, rate-limit counters
Database	Elasticsearch 8	Full-text search index and discovery engine
Database	MongoDB 7	Notification store and activity log (document DB)
Storage	AWS S3	Object storage for all images and video files
Storage	AWS CloudFront CDN	Global edge delivery of media and static assets
Infra	Docker	Service containerisation and local dev environment
Infra	Kubernetes (AWS EKS)	Container orchestration, auto-scaling, rolling updates
Infra	GitHub Actions	CI/CD: lint, test, build, push, deploy
Infra	Nginx	Reverse proxy and layer-4/7 load balancer
Infra	Prometheus + Grafana	Metrics collection, alerting, and dashboards
Infra	AWS Secrets Manager	Secure storage and rotation of credentials

6. Database Design

PostgreSQL is the primary relational database. All primary keys use UUIDs. Timestamps are stored in UTC. Frequently read counters (like_count, comment_count) are denormalised for read performance and reconciled asynchronously. The following schemas define the core tables.

6.1 users

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Auto-generated primary key
username	VARCHAR(50)	UNIQUE, NOT NULL	Unique public handle
email	VARCHAR(255)	UNIQUE, NOT NULL	Login email address
password_hash	TEXT	NOT NULL	Bcrypt hash (cost factor 12)
full_name	VARCHAR(100)	NOT NULL	Display name
bio	TEXT	NULLABLE	Short profile biography
profile_photo_url	TEXT	NULLABLE	CloudFront URL to avatar image
cover_photo_url	TEXT	NULLABLE	CloudFront URL to banner image
is_private	BOOLEAN	DEFAULT false	Private-account flag
is_verified	BOOLEAN	DEFAULT false	Email verified flag
created_at	TIMESTAMPTZ	DEFAULT now()	Account creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT now()	Last profile update timestamp

6.2 posts

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Auto-generated primary key
user_id	UUID	FK → users.id	Post author
caption	TEXT	NULLABLE	Text body or media caption
post_type	VARCHAR(10)	NOT NULL	text photo video
visibility	VARCHAR(10)	DEFAULT 'public'	public followers private
location_tag	VARCHAR(100)	NULLABLE	Optional location label
like_count	INT	DEFAULT 0	Denormalised like counter
comment_count	INT	DEFAULT 0	Denormalised comment counter
created_at	TIMESTAMPTZ	DEFAULT now()	Post creation timestamp
updated_at	TIMESTAMPTZ	DEFAULT now()	Last edit timestamp

6.3 post_media

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Primary key
post_id	UUID	FK → posts.id	Parent post (cascade delete)
media_type	VARCHAR(10)	NOT NULL	image video
s3_key	TEXT	NOT NULL	AWS S3 object key
cdn_url	TEXT	NOT NULL	CloudFront delivery URL
width	INT	NULLABLE	Width in pixels
height	INT	NULLABLE	Height in pixels
duration_sec	FLOAT	NULLABLE	Video duration (seconds)
order_index	INT	DEFAULT 0	Position in carousel

6.4 comments

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Primary key
post_id	UUID	FK → posts.id	Commented post
user_id	UUID	FK → users.id	Comment author
parent_id	UUID	FK → comments.id, NULL	Parent for nested replies
body	TEXT	NOT NULL	Comment content (max 1 000 chars)
like_count	INT	DEFAULT 0	Denormalised like counter
created_at	TIMESTAMPTZ	DEFAULT now()	Comment timestamp

6.5 likes

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Primary key
user_id	UUID	FK → users.id	User who liked
target_type	VARCHAR(10)	NOT NULL	post comment
target_id	UUID	NOT NULL	ID of liked entity
created_at	TIMESTAMPTZ	DEFAULT now()	Like timestamp

6.6 follows

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Primary key

follower_id	UUID	FK → users.id	Who is following
following_id	UUID	FK → users.id	Who is being followed
status	VARCHAR(10)	DEFAULT 'active'	active pending blocked
created_at	TIMESTAMPTZ	DEFAULT now()	Follow timestamp

7. API Design

KSU CONNECT exposes a versioned RESTful API with JSON payloads. Protected endpoints require a valid Bearer JWT in the Authorization header. Base URL: <https://api.ksuconnect.com/v1>. All responses follow the envelope format { "data": {}, "meta": {}, "error": null }.

7.1 Authentication

Method	Endpoint	Auth	Description
POST	/auth/register	-	Create a new account with email and password
POST	/auth/login	-	Login, returns access token + refresh token pair
POST	/auth/oauth/google	-	OAuth2 login via Google — returns token pair
POST	/auth/refresh	-	Issue a new access token from a valid refresh token
POST	/auth/logout	Y	Revoke session and blacklist the refresh token
POST	/auth/verify-email	-	Verify email address with one-time token from email
POST	/auth/forgot-password	-	Trigger password-reset email dispatch
POST	/auth/reset-password	-	Set new password using the reset token from email

7.2 Users

Method	Endpoint	Auth	Description
GET	/users/:id	-	Get public profile of any user by ID
GET	/users/me	Y	Get full profile of the authenticated user
PUT	/users/me	Y	Update own profile (bio, photo, name, privacy)
DELETE	/users/me	Y	Schedule account for permanent deletion
GET	/users/:id/posts	-	List public posts by a given user (paginated)
GET	/users/:id/followers	Y	List followers of a user
GET	/users/:id/following	Y	List accounts a user follows
POST	/users/:id/follow	Y	Follow a user (or submit follow request if private)
DELETE	/users/:id/follow	Y	Unfollow a user
GET	/users/suggestions	Y	Get personalised follow suggestions

7.3 Posts

Method	Endpoint	Auth	Description
POST	/posts	Y	Create a new post (text, photo, or video)

GET	/posts/:id	-	Get a single post by ID
PUT	/posts/:id	Y	Edit caption or visibility of own post
DELETE	/posts/:id	Y	Delete own post and associated media files
GET	/feed	Y	Fetch personalised newsfeed (cursor paginated)
GET	/explore	-	Fetch trending / explore posts
POST	/posts/:id/like	Y	Like a post
DELETE	/posts/:id/like	Y	Remove like from a post
POST	/posts/:id/share	Y	Repost with optional quote caption

7.4 Comments

Method	Endpoint	Auth	Description
GET	/posts/:id/comments	-	List top-level comments on a post (paginated)
POST	/posts/:id/comments	Y	Add a comment to a post
GET	/comments/:id/replies	-	List replies to a comment
POST	/comments/:id/replies	Y	Reply to a comment
PUT	/comments/:id	Y	Edit own comment body
DELETE	/comments/:id	Y	Delete own comment
POST	/comments/:id/like	Y	Like a comment

7.5 Media, Search & Notifications

Method	Endpoint	Auth	Description
POST	/media/upload	Y	Upload image or video; returns S3 key and CDN URL
GET	/search/users	-	Search users by username or display name
GET	/search/posts	-	Search posts by keyword or hashtag
GET	/search/hashtags	-	List trending hashtags and their post counts
GET	/notifications	Y	Get paginated notification list for current user
PUT	/notifications/read	Y	Mark all or specific notifications as read
DELETE	/notifications	Y	Clear notification history

8. Security Considerations

Authentication & Authorisation

All protected API endpoints require a JWT Bearer token. Access tokens expire after 15 minutes; refresh tokens last 30 days and are stored in HTTP-only cookies to prevent XSS theft. Tokens are validated at the API Gateway before any request reaches a microservice. Role-based access control (RBAC) differentiates regular users from admins and moderators.

Password Security

Passwords are hashed with Bcrypt at a cost factor of 12 before storage. Plain-text passwords are never logged or included in API responses. Password strength is enforced client- and server-side: minimum 8 characters, at least one uppercase letter, digit, and symbol.

Data Encryption

All data in transit is protected by TLS 1.3 (HTTPS enforced via HSTS). Sensitive fields (email, phone) are encrypted at rest with AES-256 using AWS KMS-managed keys. AWS S3 uses server-side encryption (SSE-S3) for all stored media objects.

Rate Limiting & DDoS Protection

The API Gateway enforces per-IP and per-user rate limits via Redis-backed counters. CloudFront provides edge-level DDoS mitigation. Strict per-endpoint limits apply: e.g. login — 5 req/min; post creation — 10 req/min; media upload — 20 req/hr.

Input Validation & Sanitisation

All user input is validated server-side with Zod/Joi schemas before processing. SQL injection is prevented by ORM-enforced parameterised queries (no raw SQL). File uploads are validated for MIME type, magic bytes, and file size before any processing.

Content Moderation

Uploaded media is scanned by AWS Rekognition for explicit content before being published. Users can report posts and accounts; reports feed a human-review queue. Repeated violations trigger automated account suspension pending admin review.

GDPR Compliance

Users may request a JSON export of all personal data (Data Subject Access Request). Account deletion permanently removes all PII within 30 days. A cookie-consent banner and Privacy Policy are shown on first visit. Data Processing Agreements are in place with all third-party sub-processors (AWS, Google, Cloudflare).

Infrastructure Security

All services run inside a private AWS VPC. Only the API Gateway and CloudFront are publicly accessible. Kubernetes network policies restrict inter-service communication to declared routes only. All credentials and API keys are stored in AWS Secrets Manager and injected at runtime — never hard-coded or committed to version control.

9. Development Roadmap

The project is delivered across six phases, each building on the previous. A working MVP is available at the end of Phase 2 (week 9). The public beta launches at the end of Phase 6 (week 24).

Phase	Timeline	Deliverables
Phase 1 – Foundation	Weeks 1–4	Monorepo setup, CI/CD pipeline, PostgreSQL schema migrations, Auth Service (re
Phase 2 – Core Social	Weeks 5–9	Post Service (text + photos), Like and Comment system, Follow system, chronolog
Phase 3 – Real-time & Video	Weeks 10–14	Video upload and FFmpeg HLS transcoding, WebSocket notification server, Rabb
Phase 4 – Discovery	Weeks 15–17	Elasticsearch search service, full-text user and post search, hashtag system, trend
Phase 5 – Scale & Hardening	Weeks 18–20	Kubernetes EKS deployment, horizontal auto-scaling, security penetration test, loa
Phase 6 – Advanced & Beta	Weeks 21–24	Stories (24-hr ephemeral posts), emoji reactions, AWS Rekognition moderation, a

10. Suggested Team Structure

The following team composition is recommended for the six-phase delivery. Roles can be combined for smaller teams during early phases.

Role	Head count	Responsibilities
Product Manager	1	Roadmap, sprint planning, stakeholder alignment, user research
UI / UX Designer	1–2	Wireframes, prototypes, design system, usability testing
Frontend Developer (Web)	2–3	React / Next.js, component library, accessibility, performance
Mobile Developer	1–2	React Native app for iOS and Android
Backend Developer	3–4	Microservices, REST/GraphQL APIs, database optimisation
DevOps / Platform Engineer	1–2	Kubernetes, CI/CD, cloud infra, monitoring, cost management
QA Engineer	1–2	Test automation, regression suites, performance testing
Security Engineer	1	Security audit, pen testing, GDPR compliance, vulnerability management

11. Conclusion

KSU CONNECT is designed to be a feature-rich, scalable, and secure social media platform delivering a compelling user experience from day one. The microservices architecture ensures each component can be developed, deployed, and scaled independently, reducing risk and enabling rapid iteration across parallel teams.

The six-phase roadmap produces a working MVP by week 9, progressively adding real-time capabilities, video handling, and advanced discovery features through to a hardened public beta at week 24. With a strong emphasis on security, privacy, and observability, KSU CONNECT is built to grow with its user base

and adapt to evolving social media expectations.

This document is the foundational technical specification and should be reviewed and updated at each major phase milestone to reflect implementation decisions, revised estimates, and evolving product requirements.